

Séance 8

Définitions

ConfigMaps

Un [ConfigMap](#) est une ressource Kubernetes qui permet de stocker des données de configuration sous forme de paires clé-valeur et de les découpler du code des applications. Ce type de ressource est utilisé pour rendre les applications conteneurisées plus portables et plus faciles à configurer selon l'environnement, en fournissant aux conteneurs les paramètres nécessaires via des variables d'environnement ou en montant des fichiers de configuration. Cela permet de séparer le code des configurations, facilitant ainsi les mises à jour et l'adaptation à différents environnements (développement, production, etc.).

Les cas d'utilisation principaux des *ConfigMap* :

- stockage de configurations : les *ConfigMaps* stockent des données de configuration non sensibles, comme des adresses de services externes, des ports ou des niveaux de journalisation;
- paires clé-valeur : elles permettent de stocker des données sous forme de paires clé-valeur, où la valeur peut être une chaîne simple ou un fichier de configuration complet;
- découplage du code : en séparant la configuration du code, les *ConfigMaps* permettent de garder les images de conteneurs plus légères et réutilisables;
- portabilité : les applications sont plus faciles à déployer dans différents environnements en modifiant simplement la *ConfigMap* correspondante sans avoir à reconstruire l'image du conteneur.

Les *ConfigMap* s'utilisent de différentes manières :

- via des variables d'environnement : vous pouvez configurer les conteneurs d'un pod pour utiliser les valeurs d'une *ConfigMap* comme variables d'environnement;

- via des fichiers montés : vous pouvez monter un volume basé sur une *ConfigMap* dans le système de fichiers d'un conteneur. Les données de la *ConfigMap* seront alors accessibles sous forme de fichiers;
- via des manifestes : les *ConfigMaps* sont créées et gérées à l'aide de manifestes *YAML*, tout comme d'autres objets Kubernetes.

Secrets

Un [Secret](#) est une ressource Kubernetes utilisée pour stocker des informations sensibles comme des mots de passe, des clés d'authentification ou des chaînes de connexion à des bases de données. Ce type de ressource permet de séparer les données confidentielles des configurations de Pods, rendant l'application plus sécurisée et flexible en évitant d'avoir à stocker ces informations sensibles en clair dans les fichiers de configuration ou le code source. Les secrets sont stockés de manière encodée (souvent en base64) et sont mis à disposition des applications uniquement lorsqu'elles en ont besoin.

Les cas d'utilisation principaux des *Secret*:

- paires clé-valeur : chaque secret est une paire clé-valeur où la clé est un nom (par exemple, `username`) et la valeur est l'information sensible correspondante (par exemple, `secret password`);
- stockage sécurisé : les secrets sont stockés dans le cluster Kubernetes, mais de manière encodée pour éviter les fuites d'informations sensibles;
- utilisation par les applications : les secrets peuvent être montés dans des Pods comme des fichiers ou utilisés comme des variables d'environnement, permettant aux applications de les lire et de les utiliser pour s'authentifier auprès d'autres services;
- séparation des responsabilités : en externalisant les secrets, les développeurs peuvent se concentrer sur le code de l'application sans avoir à gérer directement les informations d'identification sensibles dans les fichiers de configuration.

Les *Secret* s'utilisent de différentes manières :

- via des variables d'environnement : vous pouvez configurer les conteneurs d'un pod pour utiliser les valeurs d'un *Secret* comme variables d'environnement;
- via des fichiers montés : vous pouvez monter un volume basé sur un *Secret* dans le système de fichiers d'un conteneur. Les données du *Secret* seront alors accessibles sous forme de fichiers;
- via des manifestes : les *Secret* sont créés et gérés à l'aide de manifestes *YAML*, tout comme d'autres objets Kubernetes.

Tâche 1

De manière déclarative, créez un *Deployment* exposant la variable d'environnement **BG_COLOR** via un *ConfigMap*.

Exemple d'un *ConfigMap* pour des variables d'environnements.

yaml

```
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config-map
data:
  VAR: value
```

Exemple de *Pod* exposant des variables d'environnement en provenances d'un *ConfigMap*

yaml

```
---
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: my-container
      image: nginx
      envFrom:
        - configMapRef:
            name: my-config-map
```

Question

Quel est le contenu du fichier yaml pour créer le *Deployment* ainsi que le *ConfigMap* ?

Tâche 2

De manière déclarative, créez un *Deployment* permettant d'exposer la variable d'environnement **BG_COLOR** via un *Secret*.

Exemple d'un *Secret* pour des variables d'environnements.

yaml

```
---
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
data:
  VAR: value
```

Exemple de *Pod* exposant des variables d'environnement en provenances d'un *Secret*

yaml

```
---
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: my-container
      image: nginx
      envFrom:
        - secretRef:
            name: my-secret
```

Questions

Quel est le contenu du fichier yaml pour créer le *Deployment* ainsi que le *Secret* ?

Que pouvez-vous observer entre le fichier que vous avez écrit et le contenu de la meme ressource dans Kubernetes ?

Que pouvez-vous observer comme différence entre une gestion via un *ConfigMap* et via un *Secret*.

Tâche 3

De manière déclarative, créez un *Deployment* permettant d'externaliser le fichier de configuration `/app/config.yaml` dans un *ConfigMap*.

Exemple du fichier `/app/config.yaml`

```
---
# config.yaml
display:
  secrets: true      # true = afficher le bloc Secrets,
                    # false = masquer
  configmap: true    # true = afficher le bloc ConfigMap,
                    # false = masquer
```

yaml

Exemple de *Pod* utilisant un fichier de configuration en provenance d'un *ConfigMap*

```
---
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: my-container
      image: nginx
      volumeMounts:
        - name: config
          mountPath: /app/config.yaml
          subPath: config.yaml
  volumes:
    - name: config
```

yaml

```
configMap:  
  name: my-cm
```

Questions

Quel est le contenu du fichier yaml pour créer le *Deployment* ainsi que le *ConfigMap* ?

Comment pouvez-vous gérer de multiple fichiers dans un même *ConfigMap* ?

Tâche 4

De manière déclarative, créez deux *Deployment* permettant d'exposer la variable d'environnement **BG_COLOR** via un *Secret* et d'externaliser le fichier de configuration `/app/config.yaml` via un *ConfigMap*.

Exigences
Le premier <i>Deployment</i> doit avoir une couleur de fond green et le block secrets masqué.
Le second <i>Deployment</i> doit avoir ue couleur de fond red et le block configmap masqué.

L'accès à la page web de l'application Flask doit être géré via un *Ingress* ayant un subpath spécifique à chaque déploiement.

Questions

Quel est le contenu des différents fichiers yaml ?

Expliquez la logique mise en place dans ses *Deployment* (toute ressource confondue), quel est son intérêt ?

Mise à jour le: lundi 17 novembre 2025 à 08:38

Previous page
[Séance 7](#)

Next page
[Séance 9](#)

